Clock Robustness and Alarm Precision in LLM-Generated and Upstream Zephyr RTC Tests

Pranav Krishnan
The University of Texas at Austin
Austin, Texas, USA

Keshav Bhargava
The University of Texas at Austin
Austin, Texas, USA

ABSTRACT

Firmware tests often wait a fixed time for a hardware event. If a chip's clock runs slower than expected but remains within its datasheet limits, a correct program can fail its test. We compare selected and expanded upstream Zephyr real-time clock (RTC) tests with tests generated from frozen specifications. The three original upstream cases fail at 17–26 of 100 legal clock settings; three of 66 eligible generated suites fail, all base-prompt pending tests. The prompt contrast, 3/34 versus 0/32 suites, is not significant. Expanded full applications expose additional waiting failures, phase effects and cleanup cascades; a narrower H753 clock envelope produces no observed failures. In a registered 33-subject repair study, extending waits removes observed legal-clock failures but preserves weak slow-fault detection; right-sizing waits or waiting for state improves detection. Configured-time callback checks also expose alarm offsets, but the specification requested that precision. Clock robustness, generation yield and oracle strength require separate assessment.

KEYWORDS

embedded testing, flaky tests, interrupt-driven firmware, RTC, large language models

1 INTRODUCTION

An RTC alarm test may schedule an event ten calendar seconds ahead, wait thirteen simulated seconds, and assert that a callback arrived. This is justified only if the RTC clock and delivery path fit that window. Repeating a deterministic simulator schedule cannot test that premise. Varying the oscillator within its documented range can change a verdict while preserving test and firmware source.

We call developer-written Zephyr cases *upstream tests*. Our original comparison selected three cases with visible timing windows; a later expansion executes complete qualified applications and retains capability skips. Generated tests were written from scratch from task specifications derived from the original cases' documented purpose, without supplying upstream source. They cover only those three tasks, so expansion does not establish comparability across all developer tests. We ask: **RQ1**, which tests change verdict under legal clock settings? **RQ2**, which faults do they detect, and how do mechanical repairs change that? **RQ3**, where does a manually source-derived clock threshold predict failures? **RQ4**, which timing assumptions remain in source? We contribute frozen schedules, replay/reference attribution, expanded case accounting, distribution sensitivity, and a paired repair study. Sampled-setting shares describe simulator inputs, not field failure probabilities.

2 APPROACH

Clock model and distributions. The pinned Zephyr revision [19] uses the low-speed internal oscillator (LSI) for the STM32F4 Discovery RTC. The STM32F407 datasheet characterizes 17–47 kHz at 3 V and −40 to 105 °C [14]. With the build's fixed 128×250 prescaler, ten RTC seconds ideally take $t(f) = 320{,}000/f$ Renode virtual seconds: 6.809–18.824 s. For a fixed window T, the ideal failure boundary is $f < 320/T$ kHz. Thus the running example's 13-second wait is too short below 24.615 kHz, before additional delivery effects. Thresholds were manually derived from source and frozen before runs; they apply only where one scalar clock budget is defensible.

We lead with boundaries or phase envelopes, then integrate illustrative distributions: uniform 17–47 kHz, and normals centered at 32 kHz with standard deviations 5 and 7.5 kHz (range at $\pm 3\sigma$ and $\pm 2\sigma$). Reported normal shares condition on the legal interval; the artifact also supplies untruncated values and tail mass. These are assumptions. ST discusses process, temperature, voltage, oscillator accuracy and LSI measurement in AN5067, AN5126 and RM0090 [20–22]; the reviewed evidence supplies no usable STM32F407 LSI population probability model.

Fault probes and repairs. Separate probes use out-of-spec 64-kHz and 10-kHz RTC inputs, a nominal-frequency *nonarrival fault* suppressing callback/pending delivery or freezing counting reads, and nominal-clock alarm-compare shifts of −1 or +1 RTC second. A *kill*, used in tables, means an activated fault produces an explicit selected-case assertion failure. Programming correctness, when the RTC reaches an event, and callback delivery are distinct properties.

Mechanical repairs preserve original checks and use a 20.824-second experimental ceiling (18.824 plus a 2-second margin). V0 is unchanged; V1 sets the wait to max(original, 20.824) and never shortens it; V1b sets it to exactly 20.824. V2 waits on the existing event/state predicate with 20-ms polling under that ceiling. V3 adds missing checks required by the frozen specification, including alarm readback and callback calendar equality. The margin is not a measured bound on delivery latency.

3 EXPERIMENTAL SETUP

Generation and screening. `gpt-6-luna`, `gpt-6-sol`, and `gpt-6-astra` (Luna, Sol and Astra) each supplied five independent source-only C tests using ztest (Zephyr's test framework), for 13 tasks and two prompts: 390 generations. The *base prompt* omits numeric clock limits; the *prompt with clock ranges* adds LSI and high-speed internal oscillator (HSI) ranges plus a callback-delivery reminder. The other ten tasks concern narrower HSI timing effects and are outside this RTC/fault comparison. Models had no tools, execution

or feedback. Functional review, build and five nominal passes determine eligibility; timing adequacy is measured separately. Across all tasks, 330/390 passed functional review and 321/390 nominal screening. The RTC cohort has 90 attempts and 66 eligible suites.

The callback prompt's required checks include:

> "Neither callback is observed before the alarm time. Both callbacks fire at the configured RTC time and report the expected alarm IDs."

The pending prompt includes:

> "The test configures and reads back a supported RTC alarm time successfully. The alarm is not observed pending before its configured RTC time."

Its remaining check requires eventual pending state after the target under a finite, justified bound. Both prompt conditions use the same task checks.

Execution and attribution. A fresh Renode process (an instruction-level embedded simulator) [2] runs each of 100 shared sampled integer frequencies and both endpoints. Source/ELF, CPU, SysTick and synchronization quantum stay fixed. All 66 generated suites are now measured, including the 16 formerly forecast-only suites. The task is the comparison unit; registered contrasts subtract base from range-prompt rates within model/task, then average tasks and models equally. *Hybrid bounds* originally combined measured verdicts, frozen forecasts and missing-outcome extremes; no unrun forecasts remain, but missing-outcome bounds still apply. Fault probes cover 66 generated and three isolated upstream cases; alarm offsets activate in 39 generated and two upstream alarm cases. The 27 generated and one upstream counting case are controls.

A waiting-defect attribution requires a saved assertion, exact same-input replay, source explanation and passing task-specific reference checking premature behavior and eventual progress under a justified ceiling. Full applications additionally separate phase-dependent waits, cleanup cascades and unresolved rate contracts. Missing outcomes stay missing. Execution uses the calibrated Renode v1.17.0 Linux container; its 816-case calibration matched local verdicts and trace hashes. The fault/offset probes and all-attempt, paired, similarity, expansion and repair analyses were added after initial varied-clock runs began; dated amendments and immutable seals are in the repository.

Expansion and repair scope. We run every case in RTC API (7 declared), counter basic (11) and counter seconds (1) on each qualified configuration. Counter applications use a *non-stock RTC-only configuration*, disabling the STM32 timer counter and RTC API driver while enabling the RTC counter; capability predicates remain active. F4, F767 and F746 use 17–47 kHz. H753 uses 29.760–33.600 kHz from DS12117 Rev 10, revision V, VDD 1.62–3.6 V, junction −40 to 105 °C [23]. F767 is an F4 configuration replica; all configurations share Renode's STM32F4 RTC implementation. F746 counter basic is excluded after an MPU fault, F072 after no test output, and watchdog after no reboot; no F072 divider finding is claimed. Each retained configuration passes build, five nominal runs and endpoint/setting calibration before production. The repair subset is the preselected 30 generated scalar-budget suites (27 counting, three pending) plus the three original upstream cases: no generated callback suite is in this subset.

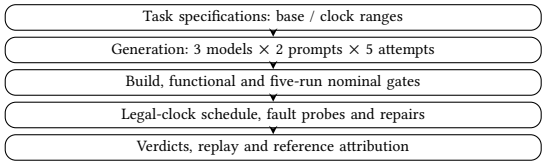


Figure 1: Generated-test evaluation. Expansion runs complete upstream applications; repairs use the frozen scalar-budget subset.

Table 1: Original isolated upstream cases. Boundary in kHz; failures per 100 sampled inputs. U is uniform; N3/N2 are illustrative legal-conditioned normal failing shares (%).

Case	Boundary	Fail	U	N3	N2
Callback	24.615	26/100	25.38	6.87	14.63
Counting	22.069	17/100	16.90	2.22	7.33
Pending	24.615	26/100	25.38	6.87	14.63

Table 2: Generated clock outcomes. B/R denote base/range prompts. All eligible suites are measured; failures exclude missing seed verdicts.

Model	Prompt	Eligible	Measured	Fail/evaluable
Luna	B	4	4	0/400
Luna	R	3	3	0/300
Sol	B	15	15	24/1496
Sol	R	15	15	0/1492
Astra	B	15	15	13/1500
Astra	R	14	14	0/1400

4 RESULTS

4.1 Legal-clock failures and expanded cases (RQ1)

The original upstream waits fail at legal slow settings; completing the generated cohort does not reveal additional failing suites. Across 69 measured suites, 6,900 seeded originals yield 6,782 passes, 106 assertion failures and 12 missing verdicts. All 112 seeded-plus-endpoint failures replay exactly and pass reference triage. Generated failures remain confined to three base-prompt pending suites: Sol r01 (11), Sol r03 (13), Astra r05 (13). The base/range-prompt totals are 37 failures in 3,400 slots with four missing, versus zero in 3,200 with eight missing. Suite-level flips are 3/34 versus 0/32 (two-sided Fisher $p = 0.239$); Luna has none under either prompt. Table 2 keeps all denominators visible.

Expansion produces failures in several complete applications, but not all are independent waiting defects (Table 3). There are 1,122 launches across 11 qualified board/application configurations, with 3,022 passing, 470 failing and 3,138 skipped case slots, no missing. On each wide-range configuration RTC has 3/5 runnable cases fail; F4/F767 counter basic has 2/4, and counter seconds has 1/1. H753 has no observed failure. Runnable means executed at least once; one H753 counter case is skipped nominally but runs at some settings. Capability-dependent skips never become passes.

The pending case has an ideal 24.615-kHz boundary. In full applications, callback and counting failures occupy sampled bands

Table 3: Expanded full applications, per configuration. Run/skip are distinct cases ever runnable/always skipped. F is any observed failure; A/B/C/D count single-boundary waits, phase-dependent waits, cleanup cascades and unresolved rate cases. F includes C/D.

Configuration	Run	Skip	F	F/Run	A/B/C/D
F4	10	9	6	6/10	1/3/1/1
F767 (F4 replica)	10	9	6	6/10	1/3/1/1
F746	6	2	4	4/6	1/2/0/1
H753	10	9	0	0/10	0/0/0/0

17.000–23.673 and 17.000–20.500 kHz, with passes from 23.847 and 20.701 kHz, respectively. Their source/model phase envelopes are 22.154–24.615 and 19.862–22.069 kHz. Renode calendar writes preserve the fractional count, so prior cases can change the remaining time to the target. This explanation is supported by source and reconciliation, not directly measured phase. For the same F4 case names, full-app versus isolated seed failures are pending 26/26, callback 22/26 and counting 12/17; no reverse disagreement occurs. Board replicas are not pooled as independent subjects.

The counter all-channels case waits three seconds for an alarm involving two requested ticks plus a driver tick; its phase envelope is 21.333–32.000 kHz. Its failure skips cleanup, causing a later single-shot setup failure, which is a cascade. Counter seconds expects one or two ticks across a 1.2-second sleep: its observed fail/pass bands are 17.000–26.168 / 27.086–47.000 kHz, but its rate reference also fails while progress passes. These rate outcomes remain firmware/simulator or accuracy-contract investigations. All 470 failing slots have references: 277 waiting slots pass, 88 clean-state cascade slots pass, and 105 rate references fail. Exact lines, per-case explanations and distribution bounds are retained in the artifact.

4.2 Fault detection and mechanical repairs (RQ2)

Surviving legal clock settings does not imply a strong fault oracle. The slow 10-kHz input takes 32 simulated seconds to reach the target: the running example's 13-second wait detects it, while a 60-second deadline may not. Every original upstream and generated suite accepts the fast oscillator, which requires an independent time reference to detect. Table 4 separates other faults. No suite above twice the legal slow bound detects the slow fault (0/12), versus 18/18 assessable suites at or below it; this 2× grouping is descriptive. Wait-style detections are 39/53 fixed-sleep-only, 7/9 event-only and 3/4 mixed.

The alarm-offset contrast is illustrative: generated callbacks detect most offsets, but their specification explicitly requests configured-time firing (Table 5). All generated pending cases detect both signs through readback-heavy checks. Across original tasks, 73 detections replay: 72 first identify compare/calendar checks and one a composite error flag. Four late-surviving Sol callbacks accept target second 5 or 6 (base r01/r02), or lack an upper calendar bound (range r01/r05); r05 additionally allows a 60-second deadline. Readback establishes compare programming, not callback-delivery precision.

Table 4: Original oscillator/nonarrival kills with descriptive 95% Wilson intervals. Missing originals are excluded; the Luna B nonarrival fault has one missing verdict. B/R are base/range prompts.

Model	Prompt	Fast oscillator	Slow oscillator	Nonarrival
Upstream	n/a	0/3 [0,56.1]%	3/3 [43.9,100]%	3/3 [43.9,100]%
Luna	B	0/4 [0,49.0]%	4/4 [51.0,100]%	3/3 [43.9,100]% +1 missing
Luna	R	0/3 [0,56.1]%	0/3 [0,56.1]%	3/3 [43.9,100]%
Sol	B	0/15 [0,20.4]%	13/15 [62.1,96.3]%	15/15 [79.6,100]%
Sol	R	0/15 [0,20.4]%	9/15 [35.7,80.2]%	15/15 [79.6,100]%
Astra	B	0/15 [0,20.4]%	9/15 [35.7,80.2]%	15/15 [79.6,100]%
Astra	R	0/14 [0,21.5]%	14/14 [78.5,100]%	14/14 [78.5,100]%
All generated	both	0/66 [0,5.5]%	49/66 [62.6,83.3]%	65/65 [94.4,100]% +1 missing

Table 5: Original nominal-clock alarm-offset kills; descriptive Wilson intervals for generated suites. One generated callback original per sign and the upstream pending early original are missing and excluded.

Case	−1 RTC s	+1 RTC s
Upstream callback	0/1	0/1
Generated callback	18/18 [82.4,100]%	14/18 [54.8,91.0]%
Upstream pending	missing	1/1
Generated pending	20/20 [83.9,100]%	20/20 [83.9,100]%

Table 6: Paired repairs on 33 scalar-budget subjects. Seed F/M are failures/missing among 3,300 legal seed slots per variant; endpoint failures are 6/66 for V0 and 0/66 for all repairs. Fault columns are kills/evaluable; Early/late are alarm offsets, applicable to five subjects. Nonarrival has one V0 missing, and fast/slow each one V1b missing. Median duration uses evaluable seeded virtual time, excluding diagnostics.

Variant	Seed F	M	Fast	Slow	Nonarrival	Early	Late	Median s
V0	106	9	0/33	21/33	32/32	4/5	4/5	11.0
V1	0	7	0/33	21/33	33/33	4/5	4/5	11.0
V1b	0	5	0/32	32/32	33/33	4/5	4/5	11.0
V2	0	3	0/33	33/33	33/33	4/5	4/5	11.0
V3	0	2	0/33	33/33	33/33	5/5	5/5	11.0

Table 7: Two axes in the original measured cohort: legal-clock result versus slow-fault detection. Unknown means missing clock verdicts, not a pass. Expanded full-application cases have no measured slow-fault result.

Cohort	Clock result	Detect slow	Accept slow
Generated	100 passes	39	16
	Observed flip	3	0
	Unknown	7	1
Isolated upstream	Observed flip	3	0

Table 6 shows why extending and right-sizing waits differ. V1 eliminates observed legal failures but leaves slow-fault detection at 21/33 because already-long waits stay long. V1b detects the slow fault in all 32 evaluable subjects; V2/V3 do so in 33/33. Only V3 makes the upstream callback detect both signed offsets. All seeded medians are 11.0 simulated seconds; these cohort medians hide subject-level differences. The 29 finalized E missing outcomes stay missing; none is replaced by a successful diagnostic. All 458 E failures replay exactly and all 112 legal-failure reference links pass.

Upstream callback, `test_alarm_callback.c`, lines 120, 126–127, 132, 140–141:

```
k_sleep(K_SECONDS(RTC_TEST_ALARM_TEST_NOT_CALLED_DELAY));
zassert_equal(callback_called_mask_status_odd, 0,
              "Alarm callback called prematurely");
k_sleep(K_SECONDS(RTC_TEST_ALARM_TEST_CALLED_DELAY));
zassert_equal(callback_called_status, true,
              "Alarm %d callback should have been called", j);
```

LLM-generated Astra base-prompt callback r01, `test.c`, lines 68 and 136–140:

```
o->time_result = rtc_get_time(dev, &o->time);
zassert_true(calendar_compare(&o->time, &target) >= 0,
             "Alarm %u fired prematurely", (unsigned int)i);
zassert_equal(calendar_compare(&o->time, &target), 0,
              "Alarm %u fired outside the target RTC second",
              (unsigned int)i);
```

Figure 2: Literal, noncontiguous excerpts from the pinned upstream callback case and saved Astra base-prompt callback r01. The first checks a fixed window; the second checks the calendar captured at callback against the target.

Table 8: All-attempt yield per 15 attempts. W: working; R: confirmed clock robustness; U: optimistic ceiling allowing missing outcomes to pass; +2: R plus slow/nonarrival kills. U is not a confidence interval.

Model	Prompt	W	R	U	+2
Luna	B	4/15	4/15	4/15	3/15
Luna	R	3/15	3/15	3/15	0/15
Sol	B	15/15	10/15	13/15	8/15
Sol	R	15/15	10/15	15/15	5/15
Astra	B	15/15	14/15	14/15	8/15
Astra	R	14/15	14/15	14/15	14/15

4.3 Threshold prediction and generation yield (RQ3)

For tests with one clear waiting deadline, the simple clock model matched every observed verdict we measured. The manually source-derived threshold applies to 30/66 eligible suites; all 30 are now measured. Three nontrivial vectors match exactly, including 37/37 failures, while the other 27 predict no failures because their budgets exceed the legal slow-arrival bound. Overall agreement is 3,000/3,000; this does not validate a threshold for the other 36 suites.

Of 90 RTC attempts, 15 fail to build, eight fail functional review, and one fails nominal screening. Confirmed robustness requires all 100 original seed verdicts to pass: 55/90 attempts meet it, with an optimistic missing-outcome ceiling of 63/90 (Table 8). Robustness plus slow/nonarrival detection is 38/90; additionally requiring a known, non-over-padded budget gives 14/90. Luna has no eligible alarm suite, and its range-prompt counting tests miss the slow fault. Model differences depend on eligibility and task composition, not a uniform range-prompt benefit.

4.4 Remaining source assumptions (RQ4)

Clock survival leaves delivery and ordering assumptions unresolved. The single unblinded source review marks 28/66 suites definite, 24 unknown and 14 without a flagged pattern. Among the 63 measured suites without observed clock failures, 28 are definite and 24 unknown. The 33-file upstream scan finds no explicit interrupt-to-callback latency bound. Paired McNemar details, reference-duration checks and source similarity are in the repository's analysis materials; they do not supply additional independent subjects.

5 DISCUSSION AND THREATS TO VALIDITY

The source contrast in Figure 2 separates a fixed observation window from calendar equality. The Astra callback also assumes delivery before the RTC calendar advances beyond the target second. Its detection of both offsets is useful but does not independently validate delivery latency. The generated specification asked for configured-time firing and pending readback, whereas upstream windows may support portability across boards/drivers; their intended portability contract was not independently established. Thus these results compare selected/expanded developer cases with generated tests under a frozen specification, not inherent superiority of model authorship.

The static board survey finds 106 RTC-enabled devicetree targets selecting LSI versus 40 selecting LSE (external crystal); these are targets, not independent devices. Yet all expanded configurations share an RTC model, and F767 is an F4 configuration replica. H753's null is a narrower modeled input-range contrast, not independent hardware validation. Counting uses a non-stock configuration. Calendar phase and prior-case cleanup alter full-application verdicts, and capability predicates can themselves depend on frequency. The rate-reference failures are retained investigations; no passing progress check substitutes for accuracy.

Uniform and normal input models are sensitivity analyses, not measured silicon distributions. Oscillator bounds do not bound ISR, callback or scheduling delay. The one-tick latency pilot failed reference feasibility; no dynamic latency verdicts or board confirmation are available. Three original generated tasks, a selected repair subset, related suites and reused frequencies limit inference; Wilson intervals and Fisher tests are descriptive. Twelve core RTC and 29 E missing verdicts remain missing. Source/padding labels have one unblinded reviewer, without inter-rater agreement. Backend model snapshots and training overlap are unavailable. The range prompt includes a generic delivery reminder, and the task specification requests precision: without reminder-only and looser-spec controls, those effects cannot be separated.

6 RELATED WORK

Asynchronous waits cause flaky tests [11]; FlakeRake injects pauses [12], and Strandberg et al. study embedded assumptions [15]. Berndt et al. study generated test flakiness [5]. Mutation and oracle surveys distinguish fault detection from correct judgments [4, 10]. ChatUniTest, CoverUp and MuTAP generate tests with LLMs [1, 6, 7]; *All Smoke, No Alarm* studies oracle signals [3]. EmbedAgent and EmbC-Test address embedded development/testing [9, 18]. Interrupt analyses include Regehr, SDRacer, AIM and AidFuzzer [8, 13, 16, 17]. We keep test binaries fixed while varying a documented peripheral-clock interval and report fault detection separately.

7 CONCLUSION AND FUTURE WORK

Legal clock variation exposes waiting assumptions, phase effects and cleanup cascades in selected/expanded upstream cases and three generated pending suites. Mechanical extension removes observed flips without fixing every oracle; right-sized or state-based waits improve slow-fault detection, and explicit calendar checks improve alarm precision. Bound waits by the worst-case legal clock with a justified delivery margin, check premature arrival against an independent clock, and assert the RTC target rather than eventual arrival alone.

Planned follow-up. The frozen, deferred callback design adds looser upstream-style and reminder-only prompts alongside fresh base/range controls, five suites per model and condition. A same-spec human packet has no participants or results. These controls would separate requested from model-chosen precision and numeric clock information from a generic reminder. Board timing, measured delivery variation and independent source coding remain future work; the LLM repair arm is not included.

AI USE

LLMs generated the evaluated tests. Automated tooling assisted execution, auditing and manuscript preparation under author direction; the authors review the evidence and take responsibility for the claims.

CODE AND DATA AVAILABILITY

Code, frozen tests, amendment log and data dictionary: https://github.com/pkrish5/zephyr-rtc-clock-robustness. Results: https://github.com/pkrish5/zephyr-rtc-clock-robustness/tree/main/results. The submission release provides hashed evaluation archives and commands to rebuild every table without Renode.

REFERENCES

[1] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE, 2897–2919. https://doi.org/10.1145/3729398

[2] Antmicro. n.d. Renode time framework documentation. Renode documentation (accessed 2026-10-13). https://renode.readthedocs.io/en/latest/advanced/time_framework.html

[3] Dipayan Banik, Kowshik Chowdhury, and Shazibul Islam Shamim. 2026. All Smoke, No Alarm: Oracle Signals in Agent-Authored Test Code. arXiv:2606.18168. https://arxiv.org/pdf/2606.18168

[4] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5, 507–525. https://doi.org/10.1109/TSE.2014.2372785

[5] Alexander Berndt, Thomas Bach, Rainer Gemulla, Marcus Kessel, and Sebastian Baltes. 2026. On the Flakiness of LLM-Generated Tests for Industrial and Open-Source Database Management Systems. In *Proceedings of the 48th International Conference on Software Engineering: Software Engineering in Practice*, 819–830. https://doi.org/10.1145/3786583.3786919

[6] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. arXiv:2305.04764. https://arxiv.org/abs/2305.04764

[7] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. *Information and Software Technology* 171, 107468. https://doi.org/10.1016/j.infsof.2024.107468

[8] Bo Feng, Meng Luo, Changming Liu, Long Lu, and Engin Kirda. 2024. AIM: Automatic Interrupt Modeling for Dynamic Firmware Analysis. *IEEE Transactions on Dependable and Secure Computing* 21, 4, 3866–3882. https://doi.org/10.1109/TDSC.2023.3339569

[9] Maximilian Harnot, Sebastian Komarnicki, Michal Polok, and Timo Oksanen. 2026. EmbC-Test: How to Speed Up Embedded Software Testing Using LLMs and RAG. arXiv:2603.09497. https://arxiv.org/pdf/2603.09497

[10] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5, 649–678. https://doi.org/10.1109/TSE.2010.62

[11] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 643–653. https://doi.org/10.1145/2635868.2635920

[12] Shanto Rahman, Aaron Massey, Wing Lam, August Shi, and Jonathan Bell. 2024. Automatically Reproducing Timing-Dependent Flaky-Test Failures. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 269–280. https://doi.org/10.1109/ICST60714.2024.00032

[13] John Regehr. 2005. Random Testing of Interrupt-Driven Software. In *Proceedings of the 5th ACM International Conference on Embedded Software*. https://doi.org/10.1145/1086228.1086282

[14] STMicroelectronics. 2026. STM32F405xx, STM32F407xx Datasheet. STMicroelectronics datasheet DS8626 Rev 12. https://www.st.com/resource/en/datasheet/stm32f407vg.pdf

[15] P. E. Strandberg, T. J. Ostrand, E. J. Weyuker, W. Afzal, and D. Sundmark. 2020. Intermittently Failing Tests in the Embedded Systems Domain. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 337–348. https://doi.org/10.1145/3395363.3397359

[16] Jianqiang Wang, Qinying Wang, Tobias Scharnowski, Li Shi, Simon Woerner, and Thorsten Holz. 2025. AidFuzzer: Adaptive Interrupt-Driven Firmware Fuzzing via Run-Time State Recognition. In *34th USENIX Security Symposium*, 1769–1786. https://www.usenix.org/conference/usenixsecurity25/presentation/wang-jianqiang

[17] Yu Wang, Linzhang Wang, Tingting Yu, Jianhua Zhao, and Xuandong Li. 2017. Automatic Detection and Validation of Race Conditions in Interrupt-Driven Embedded Software. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 113–124. https://doi.org/10.1145/3092703.3092724

[18] Ruiyang Xu, Jialun Cao, Mingyuan Wu, Wenliang Zhong, Yaojie Lu, Ben He, Xianpei Han, Shing-Chi Cheung, and Le Sun. 2026. EmbedAgent: Benchmarking Large Language Models in Embedded System Development. In *Proceedings of the 48th International Conference on Software Engineering (ICSE 2026)*. https://doi.org/10.1145/3744916.3787788

[19] Zephyr Project. 2025. Zephyr RTOS v4.3.0 (release commit 3568e1b6d5cd). Zephyr Project source revision. https://github.com/zephyrproject-rtos/zephyr/commit/3568e1b6d5cdd51a6b964a2a1d6d29200fea2056

[20] STMicroelectronics. 2018. How to optimize STM32 MCUs internal RC oscillator accuracy. Application note AN5067, Rev 1. https://www.st.com/resource/en/application_note/dm00425536-how-to-optimize-stm32-mcus-internal-rc-oscillator-accuracy-stmicroelectronics.pdf

[21] STMicroelectronics. 2023. How to calibrate internal oscillators on STM32G0 MCUs. Application note AN5126, Rev 2. https://www.st.com/resource/en/application_note/dm00469014-using-xcuberccalib-software-to-calibrate-stm32g0-series-internal-rc-oscillator-stmicroelectronics.pdf

[22] STMicroelectronics. n.d. STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs. Reference manual RM0090, Rev 21, Sec. 6.2.11. https://www.st.com/resource/en/reference_manual/dm00031020-stm32f10x-microcontrollers-stmicroelectronics.pdf

[23] STMicroelectronics. 2026. STM32H753xI Datasheet. DS12117, Rev 10, Tables 38 and 135; registered revision V envelope. https://www.st.com/resource/en/datasheet/stm32h753zi.pdf